

Handling Missing Values — Simple Guide

Missing values (NaN / null) appear when data is not recorded or unavailable. Here are the main techniques:

1. Drop Missing Values

When to use: When very few rows/columns have missing data.

python

```
import pandas as pd
```

```
df = pd.DataFrame({'Name': ['Alice', 'Bob', None], 'Age': [25, None, 30]})
```

```
# Drop rows with any missing value
```

```
df.drop_na = df.dropna()
```

```
# Drop columns with any missing value
```

```
df.dropna(axis=1)
```

2. Fill with Mean / Median / Mode

When to use: Numbers → use mean or median. Categories → use mode.

python

```
# Fill with mean
```

```
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

```
# Fill with median
```

```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

```
# Fill with mode (most frequent value)
df['Name'] = df['Name'].fillna(df['Name'].mode()[0])
```

3. Forward Fill / Backward Fill

When to use: Time series data where the previous or next value makes sense.

python

Forward fill — copies the value above

```
df['Age'] = df['Age'].ffill()
```

Backward fill — copies the value below

```
df['Age'] = df['Age'].bfill()
```

4. Fill with a Constant

When to use: When you want to mark missing values clearly.

python

Fill with a fixed value

```
df['Age'] = df['Age'].fillna(0)
```

```
df['Name'] = df['Name'].fillna('Unknown')
```

5. KNN Imputation (Smart Fill)

When to use: When you want to fill based on similar rows in your data.

python

```
from sklearn.impute import KNNImputer
```

```
imputer = KNNImputer(n_neighbors=2)
```

```
df_filled = imputer.fit_transform(df[['Age']])
```

Handling Date Format in Python — Complete Guide

1. Convert String to Date

python

```
import pandas as pd
```

```
df = pd.DataFrame({'date': ['2024-01-15', '2024-02-20', '2024-03-25']})
```

```
# Convert string to datetime
```

```
df['date'] = pd.to_datetime(df['date'])
```

```
print(df.dtypes)
```

```
print(df)
```

2. Handle Different Date Formats

python

```
df = pd.DataFrame({'date': [  
    '2024-01-15',    # YYYY-MM-DD  
    '15/01/2024',   # DD/MM/YYYY  
    '01-15-2024',   # MM-DD-YYYY  
    'Jan 15 2024',  # Month name  
    '15 January 2024', # Full month name  
    '20240115'      # Compact format  
]})
```

```
# Auto detect format
```

```
df['date'] = pd.to_datetime(df['date'], dayfirst=True, infer_datetime_format=True)
```

```
print(df)
```

3. Convert with Specific Format

```
python
```

```
df = pd.DataFrame({'date': ['15-01-2024', '20-02-2024', '25-03-2024']})
```

```
# Tell pandas exact format
```

```
df['date'] = pd.to_datetime(df['date'], format='%d-%m-%Y')
```

```
print(df)
```

Format Codes Table

Code	Meaning	Example
%Y	4-digit year	2024
%y	2-digit year	24
%m	Month number	01
%B	Full month name	January
%b	Short month name	Jan
%d	Day number	15
%H	Hour (24hr)	14
%I	Hour (12hr)	02
%M	Minutes	30
%S	Seconds	45
%p	AM or PM	PM

4. Extract Date Parts

```
python
```

```
df = pd.DataFrame({'date': ['2024-01-15', '2024-06-20', '2024-12-25']})
```

```
df['date'] = pd.to_datetime(df['date'])

df['year']    = df['date'].dt.year
df['month']   = df['date'].dt.month
df['day']     = df['date'].dt.day
df['weekday'] = df['date'].dt.day_name() # Monday, Tuesday...
df['week_no'] = df['date'].dt.isocalendar().week
df['quarter'] = df['date'].dt.quarter
df['is_weekend'] = df['date'].dt.dayofweek >= 5 # True if Sat/Sun

print(df)
```

5. Handle Errors in Date Conversion

python

```
df = pd.DataFrame({'date': ['2024-01-15', 'bad-date', None, '2024-03-25']})
```

```
# errors='coerce' → bad values become NaT (Not a Time)
```

```
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

```
# Check missing dates
```

```
print(df['date'].isnull().sum()) # count of bad/missing dates
```

```
# Fill missing dates with a default
```

```
df['date'] = df['date'].fillna(pd.Timestamp('2024-01-01'))
```

```
print(df)
```

6. Date Arithmetic (Add / Subtract Days)

python

```
from datetime import timedelta

df = pd.DataFrame({'date': ['2024-01-15', '2024-06-20']})
df['date'] = pd.to_datetime(df['date'])

# Add days
df['date_plus_10'] = df['date'] + pd.Timedelta(days=10)

# Subtract days
df['date_minus_5'] = df['date'] - pd.Timedelta(days=5)

# Difference between two dates
df['end_date'] = pd.to_datetime('2024-12-31')
df['days_left'] = (df['end_date'] - df['date']).dt.days

print(df)
```

7. Sort by Date

```
python

df = pd.DataFrame({'date': ['2024-03-10', '2024-01-05', '2024-06-20']})
df['date'] = pd.to_datetime(df['date'])

# Sort ascending
df = df.sort_values('date')

# Sort descending
df = df.sort_values('date', ascending=False)

print(df)
```

8. Filter Rows by Date

python

```
df = pd.DataFrame({'date': pd.date_range('2024-01-01', periods=10, freq='ME')})
```

Filter after a date

```
df_filtered = df[df['date'] > '2024-05-01']
```

Filter between two dates

```
mask = (df['date'] >= '2024-03-01') & (df['date'] <= '2024-07-01')
```

```
df_range = df[mask]
```

```
print(df_range)
```

9. Change Date Display Format (Output)

python

```
df = pd.DataFrame({'date': ['2024-01-15', '2024-06-20']})
```

```
df['date'] = pd.to_datetime(df['date'])
```

Change how date looks

```
df['display_date'] = df['date'].dt.strftime('%d-%m-%Y') # 15-01-2024
```

```
df['display_date2'] = df['date'].dt.strftime('%B %d, %Y') # January 15, 2024
```

```
df['display_date3'] = df['date'].dt.strftime('%d/%m/%y') # 15/01/24
```

```
print(df)
```

10. Handle Timezone

python

```
df = pd.DataFrame({'date': ['2024-01-15 10:30:00', '2024-06-20 14:00:00']})
```

```
df['date'] = pd.to_datetime(df['date'])

# Add timezone
df['date_utc'] = df['date'].dt.tz_localize('UTC')

# Convert to India timezone
df['date_ist'] = df['date_utc'].dt.tz_convert('Asia/Kolkata')

print(df)
```

Quick Reference Summary

Task	Code
String → Date	pd.to_datetime(df['col'])
Custom format	pd.to_datetime(df['col'], format='%d-%m-%Y')
Handle bad dates	errors='coerce'
Extract year	df['date'].dt.year
Extract month	df['date'].dt.month
Day name	df['date'].dt.day_name()
Add days	+ pd.Timedelta(days=10)
Date difference	(date2 - date1).dt.days
Filter by date	df[df['date'] > '2024-01-01']
Format output	dt.strftime('%d-%m-%Y')
Timezone	dt.tz_localize() / dt.tz_convert()
Resample	df.resample('ME').sum()
